STACK TECHNIQUE ET PLAN D'IMPLEMENTATION

PHASE 1 - ENVIRONNEMENT LOCAL

XAMPP - PHP MYADMIN - LARAVEL

Encyclopedie Inclusive Ivoirienne (EII)

Developpement 100% local avant mise en production

DOCUMENT TECHNIQUE COMPLEMENTAIRE

Complement au cahier des charges - Version 1.0 - 2025

INTRODUCTION

Ce document precise, de maniere operationnelle, la maniere dont la Phase 1 du projet EII (voir cahier des charges, section 7 - Roadmap et jalons) sera concretement developpee. Contrairement a la cible technologique de long terme decrite dans le cahier des charges (architecture cloud, microservices), la Phase 1 est developpee entierement en environnement local, avec des outils simples, gratuits et bien maitrises, afin de produire rapidement un prototype fonctionnel avant d'investir dans une infrastructure de production.

L'objectif de cette phase est de disposer d'une application web complete et testable sur un poste de developpement, couvrant les fonctionnalites essentielles du cahier des charges (articles multilingues, contribution, validation, medias, recherche simple, carte interactive), avant la migration vers un environnement de production distant.

> *Principe directeur de la Phase 1 : tout le developpement se fait en local, sans dependance a des services externes payants. La base de donnees est geree via phpMyAdmin (fourni par XAMPP), et l'application est construite avec le framework PHP Laravel.*

1. ENVIRONNEMENT DE DEVELOPPEMENT LOCAL

1.1 Outils de base

- **XAMPP :** distribution regroupant Apache (serveur web), MySQL/MariaDB (base de donnees) et PHP, installee localement sur la machine de developpement
- **phpMyAdmin :** interface graphique fournie avec XAMPP, utilisee pour creer, consulter et administrer la base de donnees MySQL sans ligne de commande
- **Composer :** gestionnaire de dependances PHP, necessaire pour installer et faire fonctionner Laravel
- **Laravel :** framework PHP retenu pour structurer l'ensemble de l'application (routes, controleurs, modeles, vues, migrations de base de donnees)
- **Node.js et npm :** utilises uniquement pour compiler les fichiers CSS et JavaScript du frontend (via Laravel Mix ou Vite)
- **Git :** gestion de versions du code source, avec un depot local puis distant des le debut du projet

1.2 Pourquoi ce choix pour la Phase 1

Ce choix technique privilegie la rapidite de mise en oeuvre et la simplicite de maintenance pour une petite equipe en debut de projet. Laravel est un framework mature, largement documente, qui integre nativement la plupart des besoins du projet (authentification, gestion de base de donnees via migrations, systeme de roles, envoi d'e-mails, stockage de fichiers). XAMPP permet de disposer d'un environnement complet sans configuration serveur complexe, ce qui convient bien a une phase de prototypage rapide.

2. STACK TECHNIQUE RETENUE POUR LA PHASE 1

Couche	Technologie	Role dans le projet
Serveur local	XAMPP (Apache + PHP 8.2+)	Heberge l'application Laravel en local, sur http://localhost
Base de donnees	MySQL / MariaDB (via phpMyAdmin)	Stocke articles, utilisateurs, medias, sources, validations, taxonomie
Framework backend	Laravel 11	Structure l'application : routes, controleurs, modeles Eloquent, migrations
Moteur de vues	Blade (moteur de gabarits Laravel)	Genere les pages HTML server-side (articles, formulaires, tableaux de bord)
Frontend / styles	Tailwind CSS ou Bootstrap 5	Mise en forme des pages, coherente avec la charte graphique du projet

Couche	Technologie	Role dans le projet
Interactivite legere	Alpine.js ou JavaScript natif	Bascule de langue, filtres dynamiques, previsualisation de medias
Carte interactive	Leaflet.js (charge via CDN)	Affichage de la carte de la Cote d'Ivoire et des articles geolocalises
Authentification	Laravel Breeze	Inscription, connexion, gestion des roles (contributeur, moderateur, admin)
Stockage des medias	Disque local (storage/app/public)	Conservation des images, audios et videos telecharges pendant la Phase 1
Recherche	Requetes Eloquent (LIKE / index MySQL)	Recherche simple sur les titres et contenus, suffisante pour le volume initial
Gestion de versions	Git (depot local puis distant)	Suivi des modifications du code source tout au long du developpement

Ce choix reste volontairement simple : aucun service cloud, aucune dependance payante. Le passage vers la stack cible du cahier des charges (PostgreSQL, Meilisearch, stockage objet, hebergement distant) interviendra progressivement en phase 2, une fois le prototype valide.

3. ARCHITECTURE DU PROJET LARAVEL

Le projet suit l'organisation standard d'une application Laravel, ce qui facilite la comprehension du code par tout developpeur familier du framework.

```
eii-projet/
  app/
    Models/            (Article, Contributeur, Media, Source, Validation, Categorie)
    Http/Controllers/  (ArticleController, MediaController, ValidationController...)
    Http/Middleware/   (controle des roles et permissions)
  database/
    migrations/        (creation des tables)
    seeders/           (donnees de demonstration)
    factories/         (generation de donnees de test)
  resources/
    views/             (fichiers Blade : accueil, article, admin...)
    css/ , js/         (styles et scripts compiles via Vite)
  routes/
    web.php            (routes de l'application)
  public/
    storage/           (lien symbolique vers les medias uploades)
  .env                 (configuration locale : base de donnees, cle d'application)
```

Configuration de la base de donnees (fichier .env)

La connexion a la base de donnees creee via phpMyAdmin se declare simplement dans le fichier de configuration .env a la racine du projet Laravel :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=eii_local
DB_USERNAME=root
DB_PASSWORD=
```

Une fois cette configuration en place, les migrations Laravel (commande "php artisan migrate") creent automatiquement l'ensemble des tables dans la base de donnees geree par phpMyAdmin.

4. MODELE DE DONNEES (TABLES PRINCIPALES)

Le schema ci-dessous reprend les entites definies dans le cahier des charges (section 4.2), traduites en tables MySQL et en modeles Eloquent Laravel.

Table (migration)	Champs principaux	Relations cles
users	id, nom, email, mot_de_passe, role, region, points, cree_le	Un utilisateur possede plusieurs articles et validations
articles	id, slug, statut, auteur_id, latitude, longitude, cree_le, modifie_le	Appartient a un auteur ; possede plusieurs traductions, medias et sources
article_traductions	id, article_id, langue, titre, contenu	Rattachee a un article (une ligne par langue disponible)
medias	id, article_id, type, chemin_fichier, narrateur, date_enregistrement, licence	Rattache a un article ; type = audio, video, image ou modele 3D
sources	id, article_id, type, auteur, date, langue, statut_validation	Rattachee a un article ; type = orale ou ecrite
validations	id, article_id, etape, validateur_id, commentaire, horodatage	Rattachee a un article et a un utilisateur validateur
categories	id, nom, type, parent_id	Hierarchie de categories (region, groupe ethnique, theme)
article_categorie	article_id, categorie_id	Table pivot : association entre articles et categories

5. MODULES FONCTIONNELS (COMPARTIMENTS DU PROJET)

Le developpement de la Phase 1 est organise en sept modules independants, developpes et testes separement avant leur integration dans l'application complete.

Module 1 - Authentification et roles : mis en place avec Laravel Breeze. Trois roles sont geres : contributeur, moderateur/validateur et administrateur. Chaque role determine les actions accessibles dans l'interface (redaction, validation, administration).

Module 2 - Articles multilingues : gestion des articles avec creation, edition et affichage. Chaque article peut posseder plusieurs traductions (table article_traductions), avec un selecteur de langue sur la page de consultation.

Module 3 - Gestion des medias : telechargement de fichiers (images, audios, videos) associes a un article, stockes localement dans storage/app/public et affiches via le lien symbolique public/storage genere par Laravel.

Module 4 - Workflow de contribution et validation : implementation des cinq etapes de validation definies dans le cahier des charges, gerees par le champ "statut" de l'article et la table "validations" qui trace chaque etape franchie.

Module 5 - Taxonomie et recherche : gestion des categories (regions, groupes ethniques, themes) et recherche simple sur les titres et contenus des articles, via des requetes Eloquent filtrees.

Module 6 - Carte interactive : integration de la bibliotheque Leaflet.js cote frontend, alimentee par les coordonnees latitude/longitude enregistrees sur chaque article.

Module 7 - Tableau de bord administrateur : interface reservee aux moderateurs et administrateurs, presentant la file de validation en attente, la liste des utilisateurs et des statistiques simples de contenu.

6. ETAPES D'INTEGRATION - METHODOLOGIE DE DEVELOPPEMENT

Le developpement suit un enchainement de dix etapes, chacune validee avant de passer a la suivante, afin de limiter les regressions et de garder a tout moment une version fonctionnelle de l'application.

Etape	Contenu	Livrable verifiable
1. Installation de l'environnement	Installation de XAMPP, creation de la base via phpMyAdmin, installation de Composer et de Laravel, initialisation du depot Git	Page d'accueil Laravel par defaut accessible sur localhost
2. Conception de la base de donnees	Redaction des migrations pour toutes les tables du modele de donnees (section 4)	Tables visibles et coherentes dans phpMyAdmin
3. Authentification et roles	Installation de Laravel Breeze, ajout du champ role, creation d'un middleware de controle d'acces	Inscription, connexion et acces restreint selon le role fonctionnels
4. CRUD articles et multilinguisme	Creation des controleurs et vues pour creer, modifier et afficher un article, avec gestion des traductions	Un article peut etre cree, edite et consulte en plusieurs langues
5. Gestion des medias	Formulaire de televersement, stockage local, affichage des fichiers associes a un article	Un article affiche correctement ses images, audios ou videos
6. Workflow de validation	Implementation des cinq etapes de validation et des notifications internes associees	Un article suit le parcours complet jusqu'a la publication
7. Taxonomie et recherche	Mise en place des categories, association aux articles, moteur de recherche simple	Filtrage et recherche d'articles operationnels
8. Carte interactive	Integration de Leaflet.js et affichage des articles geolocalises sur la carte	Carte fonctionnelle avec points cliquables
9. Interface d'administration	Tableau de bord recapitulant la file de validation, les utilisateurs et les statistiques de base	Un administrateur supervise l'activite depuis un ecran unique
10. Tests et donnees de demonstration	Redaction de jeux de donnees (seeders), tests manuels des parcours utilisateurs, documentation technique	Application testable de bout en bout avec un jeu de donnees realiste

7. BONNES PRATIQUES DE DEVELOPPEMENT

• **Gestion de versions :** chaque etape du developpement fait l'objet de commits Git reguliers et d'un depot distant (GitHub ou GitLab) des le debut du projet, meme en developpement local

• **Migrations versionnees :** toute modification de la structure de la base de donnees passe par une migration Laravel, jamais par une modification manuelle dans phpMyAdmin, afin de garder un historique reproductible

• **Seeders et factories :** des jeux de donnees de demonstration sont generes automatiquement pour tester l'application avec un contenu realiste sans saisie manuelle repetee

• **Fichier .env non partage :** les informations de connexion locale ne sont jamais versionnees dans Git ; un fichier .env.example documente les variables attendues

• **Respect de l'architecture MVC :** la logique metier reste dans les controleurs et modeles, les vues Blade se limitant a l'affichage

• **Tests manuels systematiques :** chaque module fait l'objet d'une verification manuelle des parcours principaux avant d'etre considere comme termine

8. LIMITES DE L'ENVIRONNEMENT LOCAL ET TRANSITION

L'environnement local presente des limites assumees pour cette phase, qui seront levees progressivement lors du passage a la Phase 2 du projet.

Limite en Phase 1 (local)	Evolution prevue en Phase 2
Base de donnees MySQL sur poste local, non accessible a distance	Migration vers PostgreSQL heberge (avec extension PostGIS pour la geolocalisation)
Stockage des medias sur disque local, sans sauvegarde automatisee	Stockage objet distant (type Cloudflare R2), avec sauvegardes programmees
Recherche simple par mots-cles (requetes SQL)	Moteur de recherche dedie (Meilisearch), tolerant aux fautes et multilingue
Application accessible uniquement sur le poste de developpement	Hebergement en ligne accessible publiquement, avec nom de domaine
Pas de canal SMS ou USSD	Integration d'un fournisseur de SMS et USSD (Africa's Talking)
Un seul environnement, sans distinction test/production	Environnements distincts de developpement, de test et de production

Ce document decrit exclusivement la mise en oeuvre technique de la Phase 1. Les choix d'architecture cible (Phase 2 et suivantes) restent ceux definis dans le cahier des charges principal, section 4 - Specifications techniques.

Stack technique et plan d'implementation - Phase 1 - Version 1.0 - 2025
Document technique complementaire au cahier des charges du projet Encyclopedie Inclusive Ivoirienne.